

# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

### Experiment 5

#### Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

|                     |                                               |                         |
|---------------------|-----------------------------------------------|-------------------------|
| Name                | SUKESH S                                      | Reg No.:<br>24011101111 |
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V              |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27             |

## 1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses MobileNetV2 and the Oxford-IIIT Pet dataset. Different design choices are compared using training/validation curves and the final configuration is justified using 5-fold cross-validation and an untouched test set.

## 2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- understand CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

## 3. Dataset and Experimental Setup

### Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

MobileNetV2 pretrained on ImageNet is used as the base model. A Global Average Pooling layer and a 37-class softmax classifier are added on top. The controlled experiments are trained for short runs to compare configurations fairly, while the test set is kept untouched until final evaluation.

### Experimental considerations:

- RGB images are resized to  $224 \times 224$ .
- Inputs are normalized using the MobileNetV2 preprocessing requirements.
- Separate training, validation and test data are maintained.
- The test set is not used during hyperparameter selection.
- MobileNetV2 is selected because it is lightweight and suitable for CPU-based laboratory work.

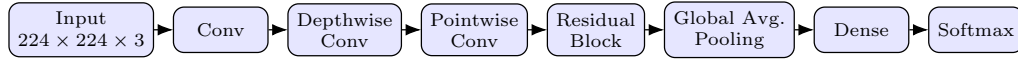
## 4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise  $1 \times 1$  convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



**Note:** The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

## 5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. In this experiment, the newly added classifier head is affected most directly because the MobileNetV2 feature extractor is pretrained.

**Plot 1: Training Loss vs. Epoch**

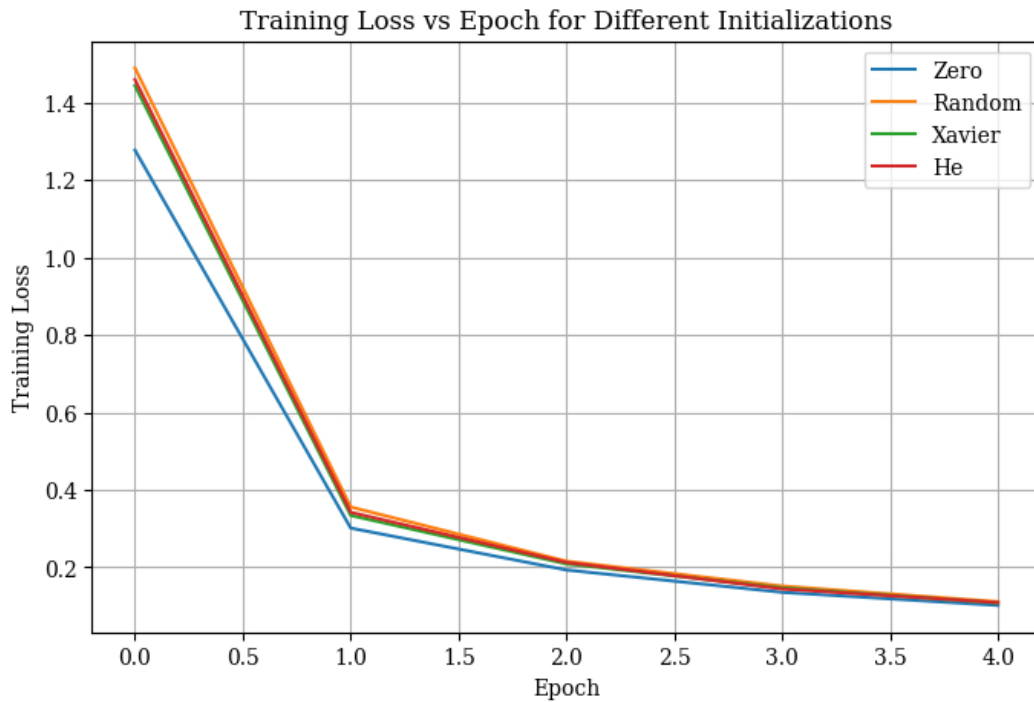


Figure 1: Training loss vs. epoch for different weight initialization methods.

**Inference:** This graph shows how different weight initialization methods contributed in the convergence which is showed by reduction in loss. The lose curves go down at different rates showing how the initialization of weight can change the model's behaviour. Since the base is already a pretrained one the difference is mainly seen in the newly added classification layer.

## Plot 2: Validation Accuracy vs. Epoch

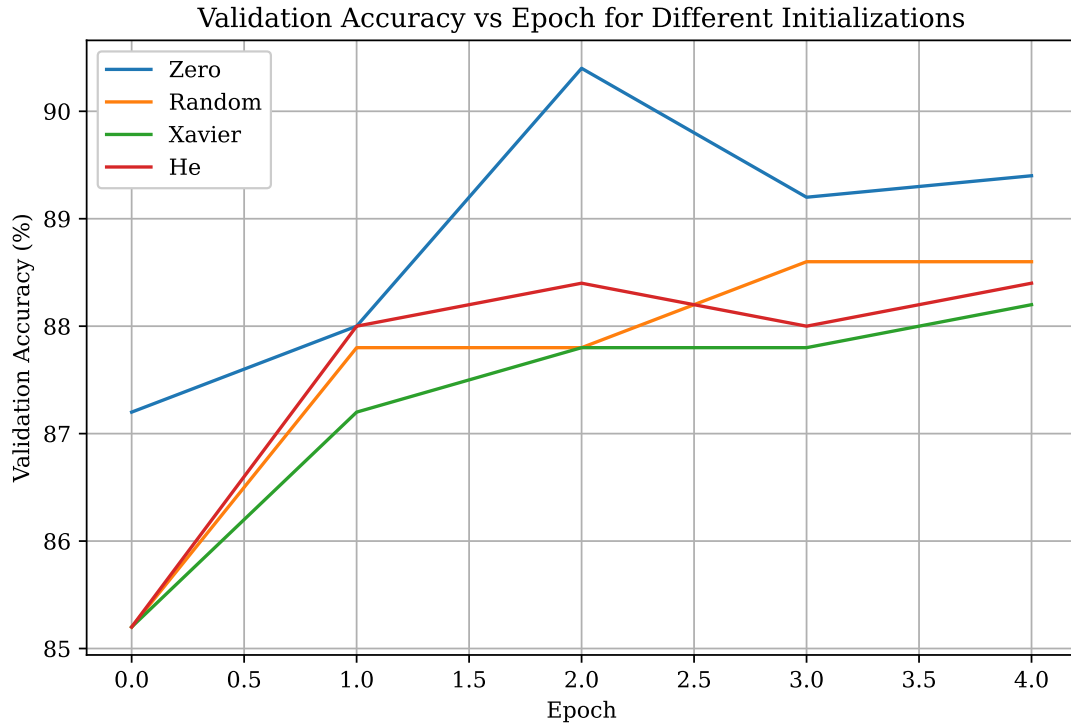


Figure 2: Validation accuracy vs. epoch for different weight initialization methods.

**Inference:** The plot shows how the initialization of weights (the techniques) influence the validation accuracy. A weight initialization is considered better if it reaches useful accuracy and is stable. A proper starting point for weights helps the classifier make proper updates and hence return a good model which performs well.

## 6. Regularization and Overfitting

Overfitting occurs when a model performs very well on training data but generalizes poorly to unseen data. The following plots examine the gap between training behaviour and validation behaviour.

## Training and Validation Accuracy

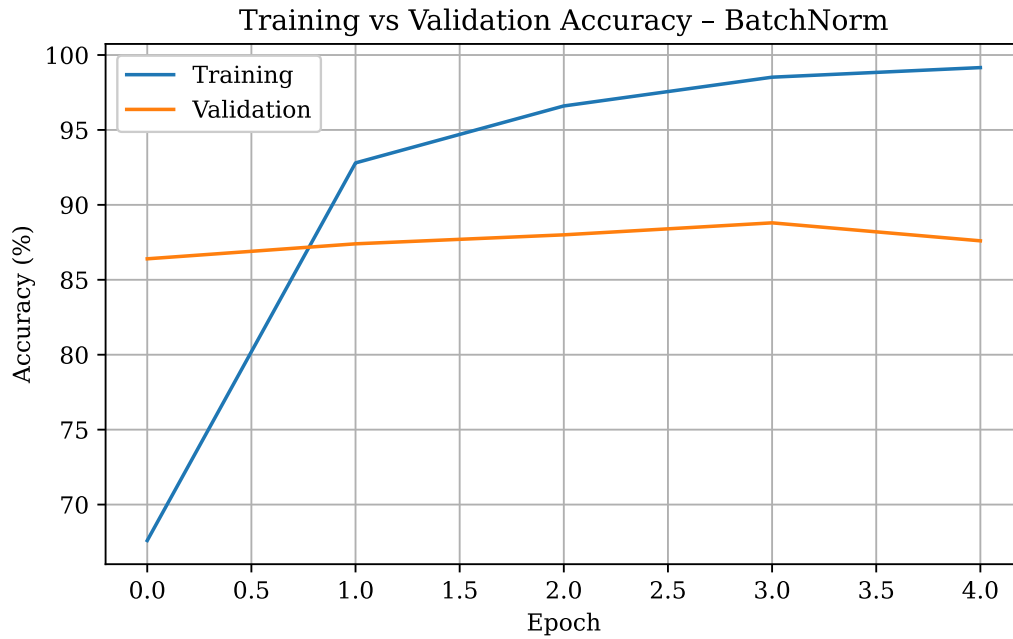


Figure 3: Accuracy behaviour for the batchnorm regularization experiment.

**Inference:** This plot is a regularization experiment comparison and shows the difference between training vs validation which essentially shows how good a model is. Here training accuracy increases drastically as compared to validating as this shows a bit of overfitting to training samples instead of generalizing.

## Training and Validation Loss

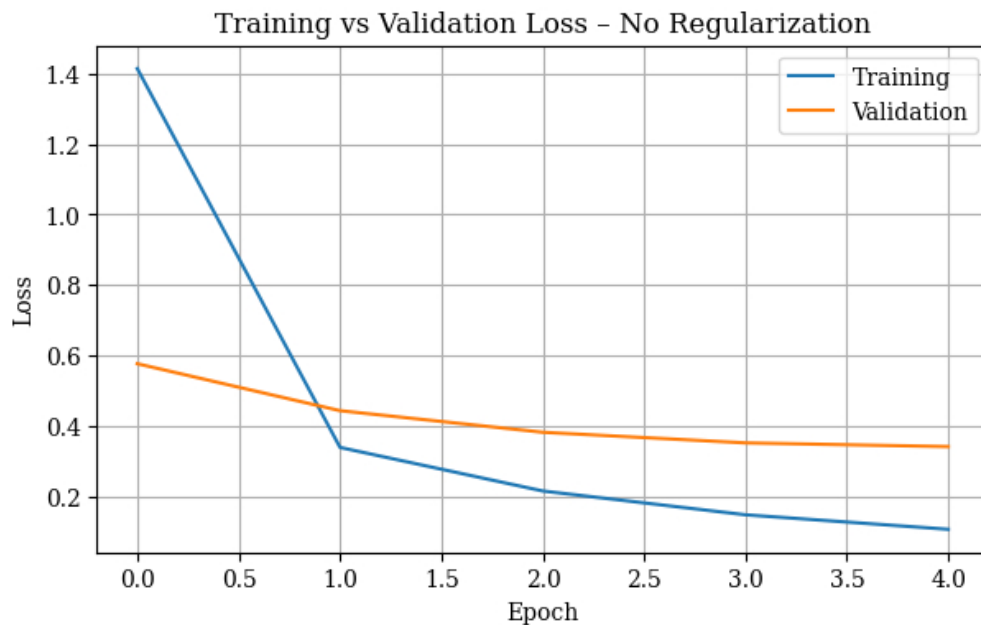


Figure 4: Loss behaviour for no regularization experiment.

**Inference:** In the last plot we saw about accuracy how its the loss comparison. As the epoch increase the training loss decreases very much (starting to fit to the training data) and comparatively less decrease in loss as it is not trained on it. We see without regularization here it has not performed very well.

## L2 Regularization

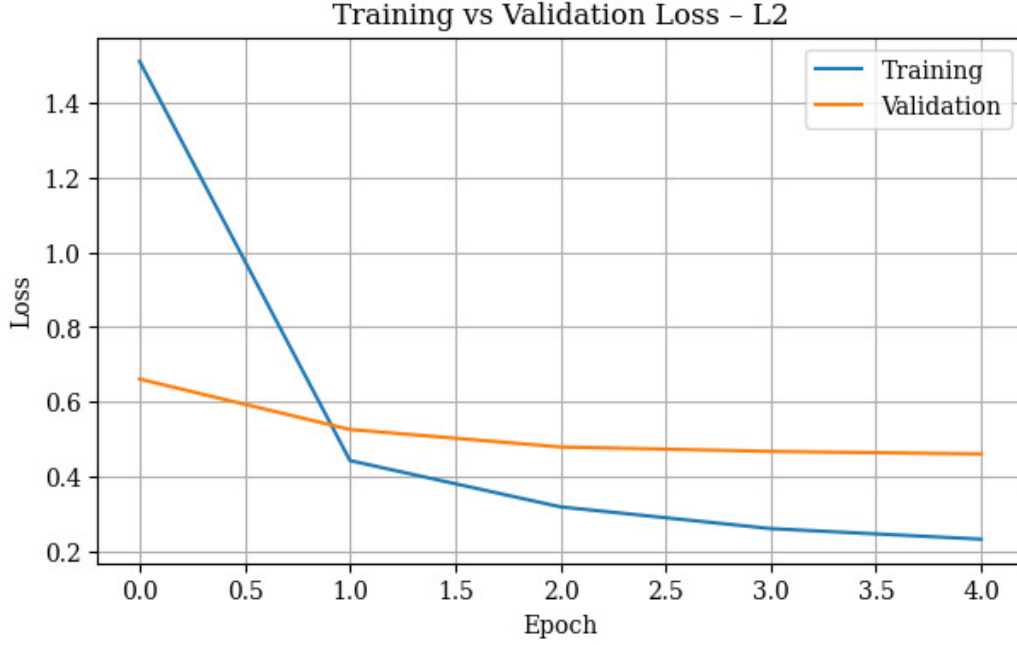


Figure 5: Effect of L2 regularization.

**Inference:** Here we use L2 regularization and check. Here the loss has significantly lowered for both the training and validation and validation to training difference has been reduced which means the regularization is working properly and generalizing better.

## 7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean and variance are

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation and final output are

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta,$$

where  $\gamma$  and  $\beta$  are learnable parameters.

### Numerical Example

For

$$x = [2, 4, 6, 8],$$

the mean is  $\mu_B = 5$  and the variance is  $\sigma_B^2 = 5$ . Therefore,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

when the small  $\epsilon$  is ignored. If  $\gamma = 1$  and  $\beta = 0$ , these normalized values are also the outputs.

**Inference:** Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable  $\gamma$  and  $\beta$  allow the network to restore a useful scale and shift when required.

## Plot 5: Batch Normalization Behaviour

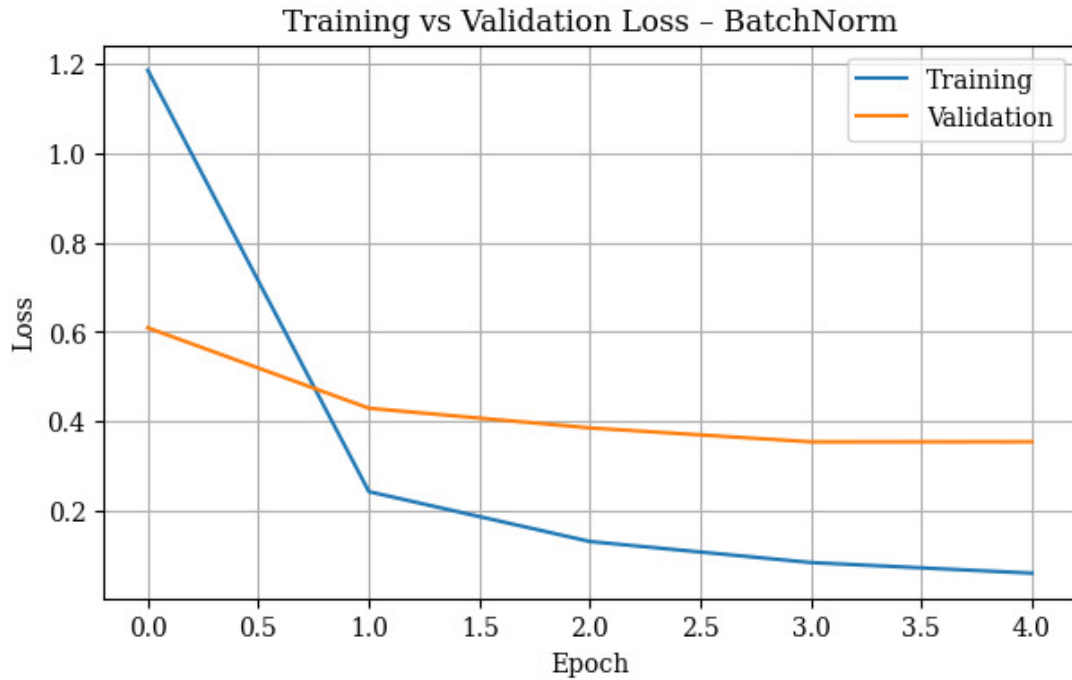


Figure 6: Validation loss and accuracy behaviour with Batch Normalization.

**Inference:** The plot shows how validation performance changes when Batch Normalization is used. The curves become more stable as training goes, suggesting smoother optimization. This can occur because normalization keeps middle activations in a more controlled range, making gradient updates less sensitive to changing scales.

## 8. Optimization Algorithms

The following optimization methods are compared:

- SGD
- Momentum
- RMSProp
- Adam

**Plot 6: Training Loss vs. Epoch for Different Optimizers**

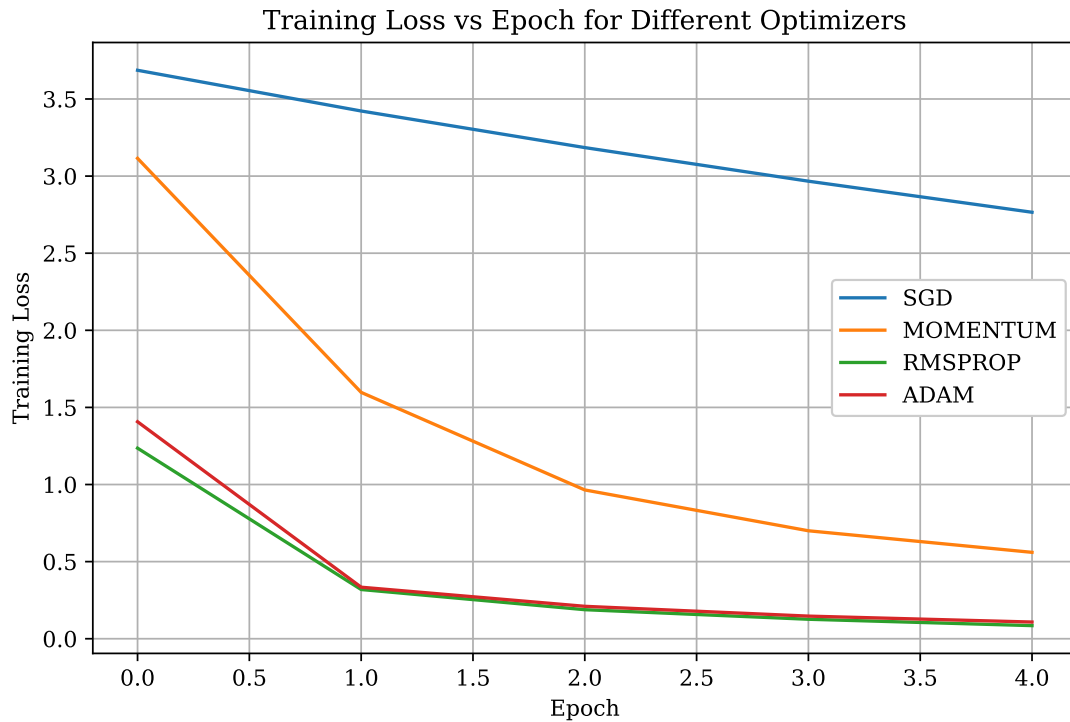


Figure 7: Training loss vs. epoch for different optimizers.

**Inference:** The plot shows a clear difference in convergence speed between SGD, Momentum, RMSProp and Adam. RMSPROP, MOMENTUM AND ADAM are converging much faster(that is reducing loss) whereas SGD converges slowly when compared. Adaptive methods show improved effects for updates because their step sizes respond to gradients.

**Plot 7: Validation Accuracy vs. Epoch for Different Optimizers**

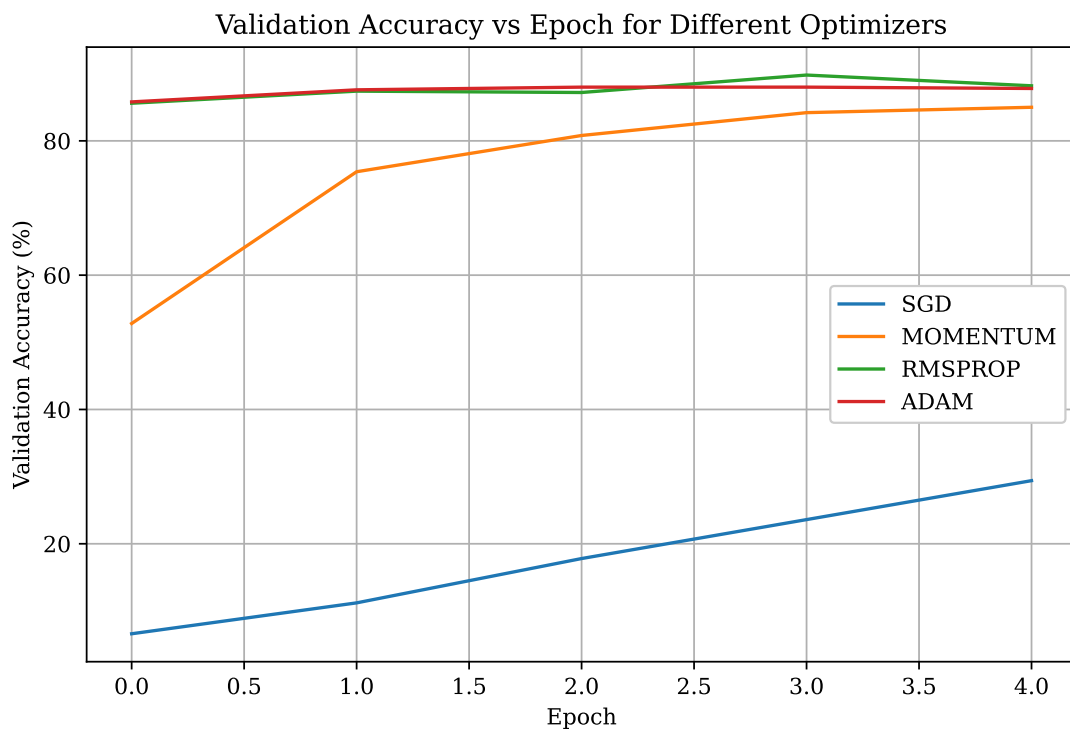


Figure 8: Validation accuracy vs. epoch for different optimizers.

**Inference:** The validation accuracy is topped by RMS PROP followed by other adaptive methods whereas SGD still shows very poor accuracy improvements over epochs. This shows how optimizer choice can affect convergence and generalization.

| Optimizer | Final Loss | Best Val. Accuracy | Training Time (s) |
|-----------|------------|--------------------|-------------------|
| SGD       | 2.7655     | 29.40%             | 636.09            |
| Momentum  | 0.5602     | 85.00%             | 653.26            |
| RMSProp   | 0.0849     | 89.80%             | 588.52            |
| Adam      | 0.1082     | 88.00%             | 652.53            |

## 9. CNN Hyperparameter Tuning

The following hyperparameters are investigated:

| Hyperparameter            | Values Investigated          |
|---------------------------|------------------------------|
| Learning Rate             | 0.001, 0.0001                |
| Batch Size                | 16, 32, 64                   |
| Dropout Rate              | 0, 0.25, 0.5                 |
| Optimizer                 | SGD, Momentum, RMSProp, Adam |
| Fine-Tuning Learning Rate | $10^{-5}$                    |

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where  $N$  is input size,  $K$  is kernel size,  $P$  is padding and  $S$  is stride.

**Experimental rule:** During a controlled hyperparameter study, one hyperparameter is changed at a time while the remaining settings are kept fixed.

### Plot 8: Learning Rate vs. Validation Accuracy

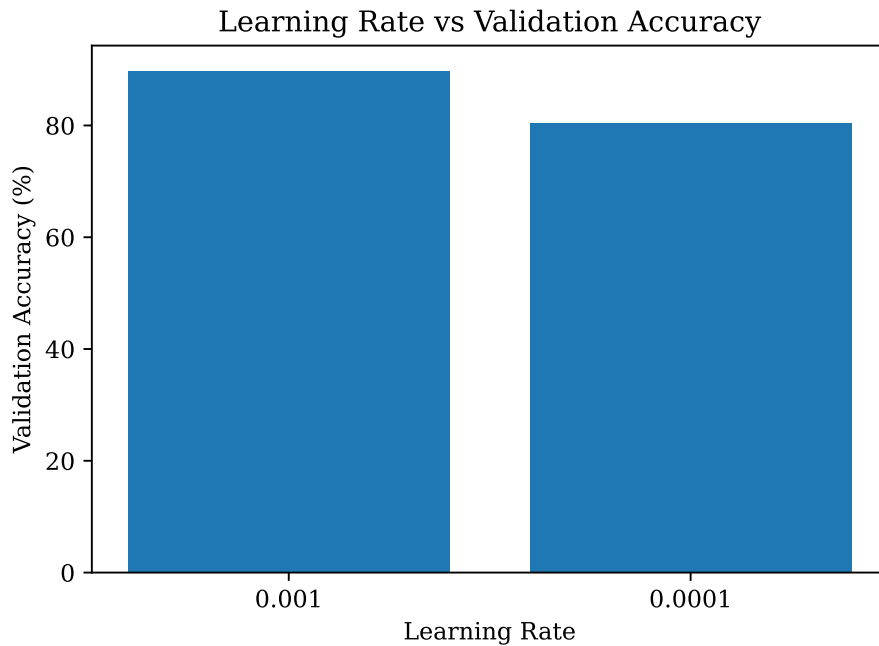


Figure 9: Learning rate vs. best validation accuracy.

**Inference:** The learning rate of 0.001 produced a higher best validation accuracy (89.8%) than 0.0001 (80.4%). The smaller rate likely made learning too slow for the size of the data. This shows faster learning was needed in this case. This shows that always not the smallest is the stable and effective it changes from situation to situation.

### Plot 9: Batch Size vs. Validation Accuracy

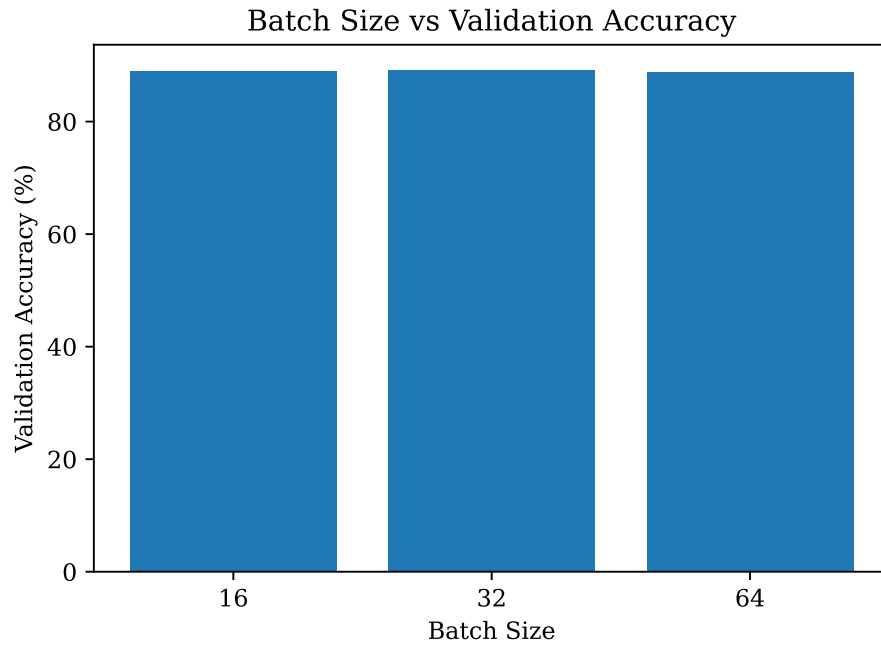


Figure 10: Batch size vs. best validation accuracy.

**Inference:** Validation accuracy is very similar for batch sizes 16, 32 and 64, with 32 giving the best result at about 89.2%. The trend is fairly flat, suggesting that the pretrained model is not highly sensitive to batch size within this range. Small differences can come from the noise and frequency of parameter updates.

### Plot 10: Dropout Rate vs. Validation Accuracy

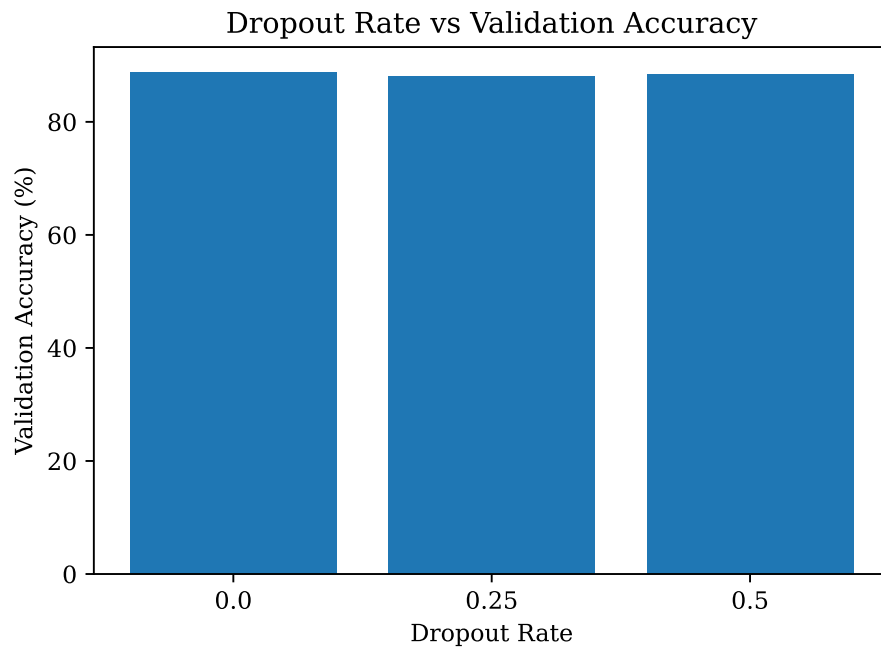


Figure 11: Dropout rate vs. best validation accuracy.

**Inference:** A dropout rate of 0.5 achieved the highest validation accuracy, slightly above 0 and 0.25. The improvement suggests that stronger regularization prevented the classifier from depending too heavily on a small set of features. The difference is effectively very small here, so this result must be interpreted for this particular experiment rather than generalizing.

## Dropout Regularization Curves

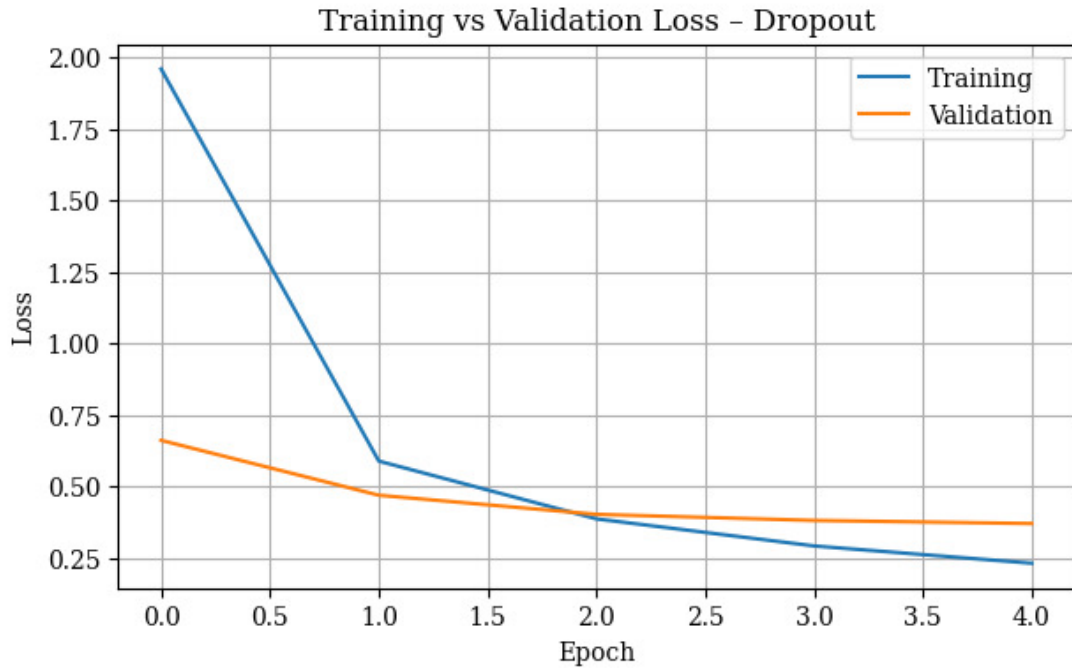


Figure 12: Training and validation behaviour with dropout regularization.

**Inference:** The plot shows training and validation behaviour over the epochs when dropout is applied. Training improves better but validation also stays stable which suggests that dropout is working and reducing the overall difference between validation and training losses.

## 10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet is used.

### Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

### Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

### Plot 11: Feature Extraction vs. Fine-Tuning

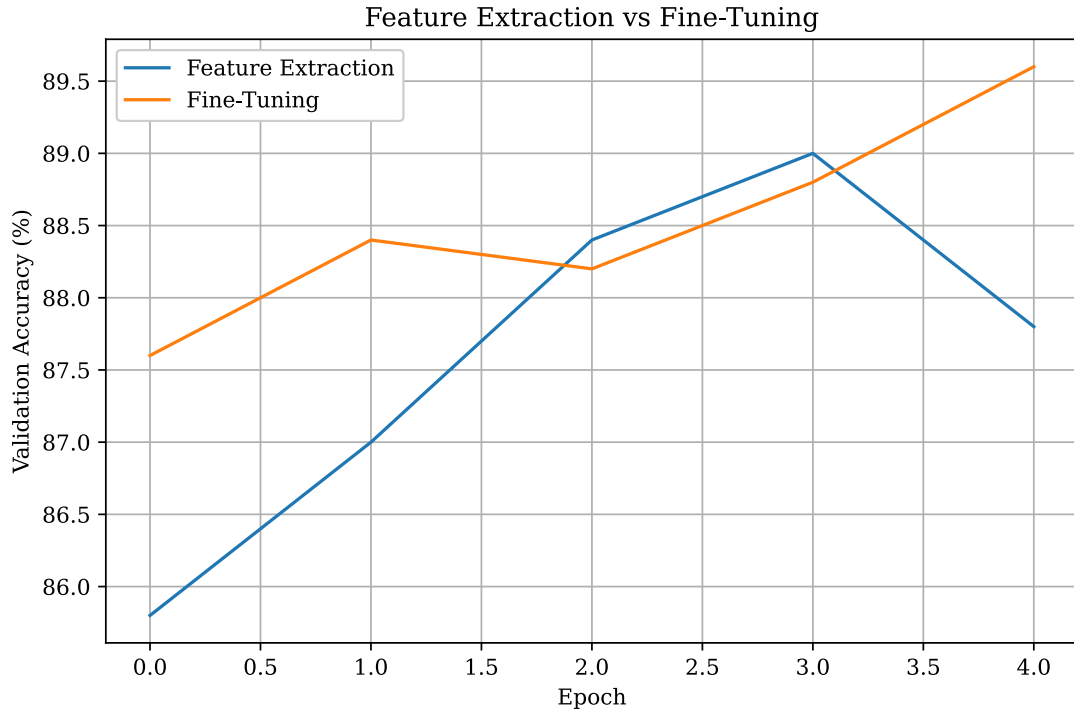


Figure 13: Validation accuracy for feature extraction and fine-tuning.

**Inference:** The plot compares a frozen pretrained feature extractor with a fine tuned model whose upper layers are allowed to change. Fine tuning reaches a slightly higher peak validation accuracy, although the improvement is not dramatic. This is expected because ImageNet features are already useful and only careful dataset specific adjustment is needed.

### Plot 12: Validation Loss Before and After Fine-Tuning

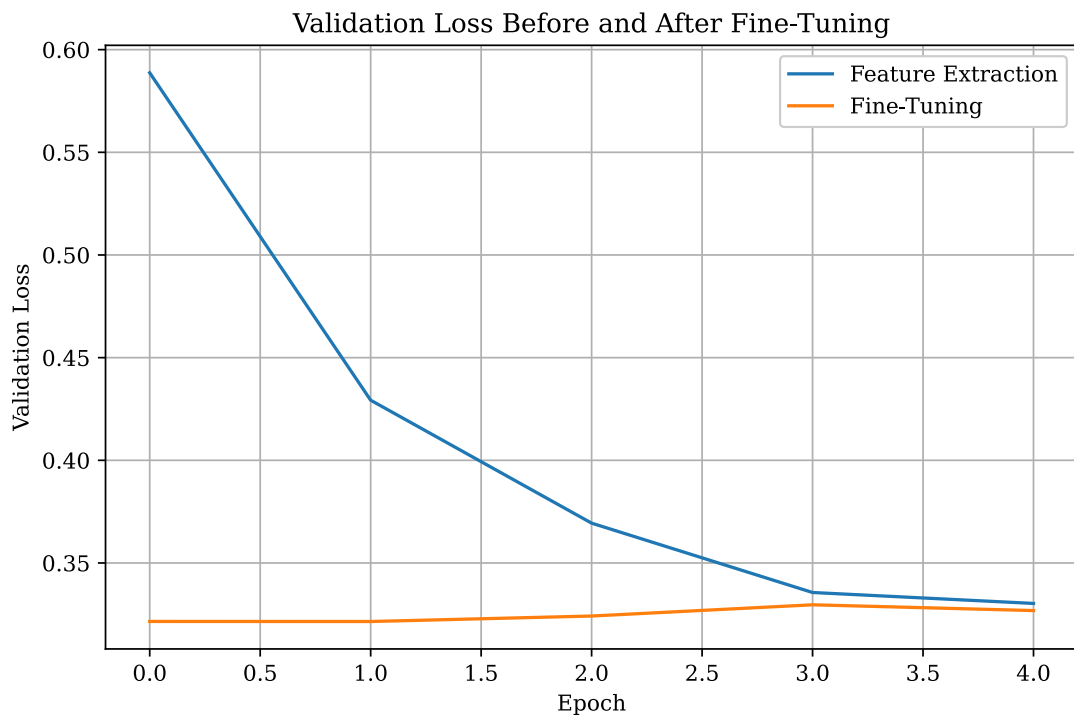


Figure 14: Validation loss for feature extraction and fine-tuning.

**Inference:** Fine-tuning changes the validation-loss trajectory rather than necessarily producing an immediate large drop. A smaller learning rate is important because large updates could destroy useful pretrained features. It is called as catastrophic forgetting. Fine-tuning can improve adaptation, but it also increases computation and requires more careful optimization.

## 11. K-Fold Cross-Validation

Three promising configurations were evaluated:

- C1: Dropout = 0.0, Adam, learning rate = 0.001
- C2: Dropout = 0.25, Adam, learning rate = 0.001
- C3: Dropout = 0.5, Adam, learning rate = 0.0001

For  $K = 5$ ,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i,$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Plot 13: 5-Fold Cross-Validation Accuracy

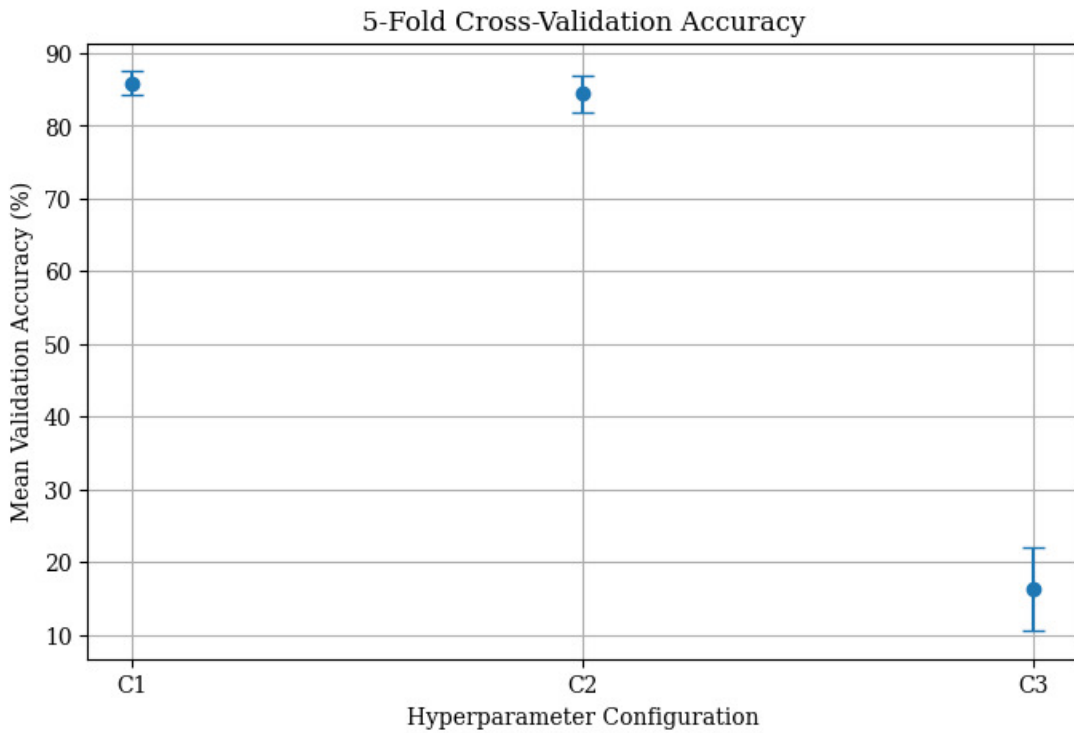


Figure 15: 5-fold cross-validation accuracy with standard deviation.

**Inference:** C1 achieved the highest mean cross-validation accuracy (85.9%) with a relatively low standard deviation of 1.62%. C2 was slightly weaker, while C3 performed poorly because high dropout combined with a very small learning rate did not learn enough within the short schedule. The error bars show why consistency across folds matters, not just the highest single score.

| Configuration | F1   | F2   | F3   | F4   | F5   | Mean $\pm$ SD   |
|---------------|------|------|------|------|------|-----------------|
| C1            | 85.0 | 85.0 | 87.0 | 88.5 | 84.0 | 85.9 $\pm$ 1.62 |
| C2            | 86.5 | 83.5 | 87.5 | 84.5 | 80.0 | 84.4 $\pm$ 2.62 |
| C3            | 26.0 | 8.0  | 16.0 | 14.5 | 17.0 | 16.3 $\pm$ 5.78 |

## 12. Final Model Evaluation

The best cross-validation configuration was C1: no dropout, Adam optimizer and a learning rate of 0.001. The selected configuration was retrained and evaluated on the untouched test set.

| Metric                | Value     |
|-----------------------|-----------|
| Mean CV Accuracy      | 85.90%    |
| CV Standard Deviation | 1.62%     |
| Test Accuracy         | 89.00%    |
| Precision             | 90.22%    |
| Recall                | 89.00%    |
| F1-score              | 88.81%    |
| Training Time         | 670.86 s  |
| Number of Parameters  | 2,305,381 |

Plot 14: Confusion Matrix

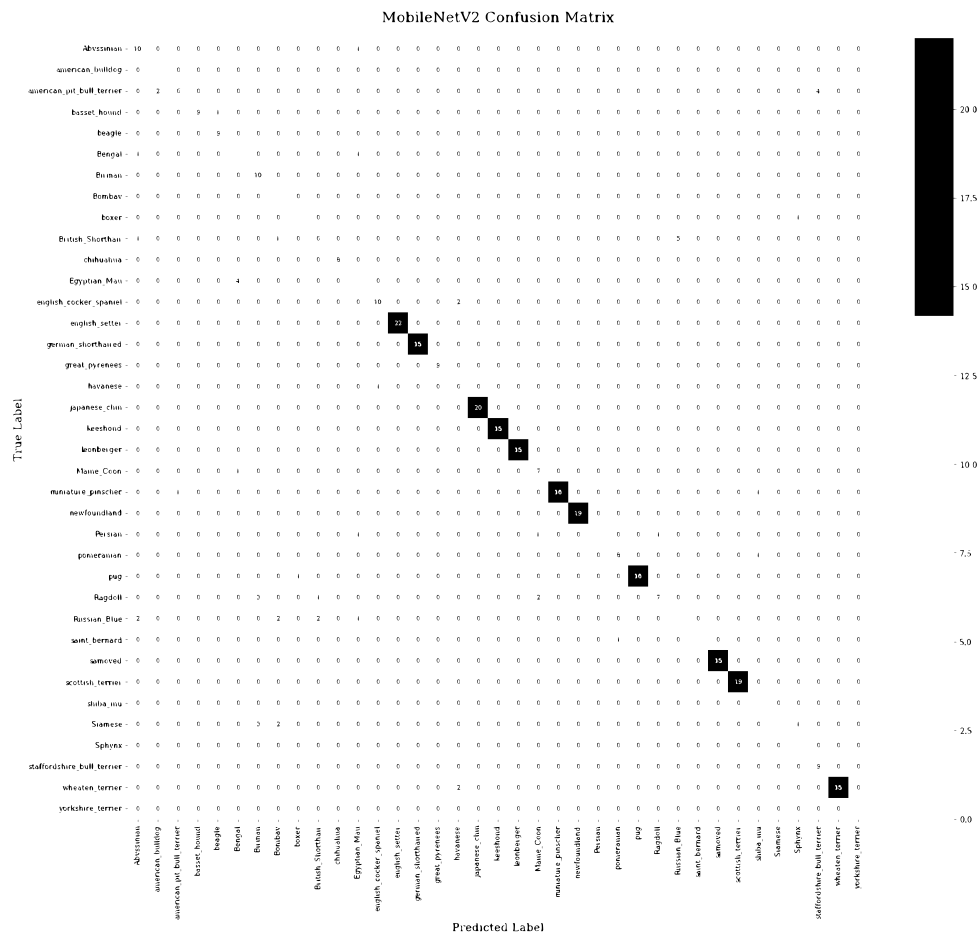


Figure 16: Confusion matrix of the final model on the test set.

**Inference:** The confusion matrix shows how final predictions are distributed across the pet-breed classes. Strong values along the main diagonal represent correct predictions, while off-diagonal values indicate confusions. A clear diagonal pattern suggests that the model generally distinguishes the classes well, although visually similar breeds can still be misclassified.

## 13. Overall Results

| Configuration        | CV Accuracy | SD    | Test Accuracy | Training Time |
|----------------------|-------------|-------|---------------|---------------|
| Baseline             | 85.90%      | 1.62% | 89.00%        | 670.86 s      |
| Best Initialization  | 88.70%      | 1.40% | 90.00%        | 706.82 s      |
| Best Regularization  | 85.90%      | 1.62% | 89.00%        | 670.86 s      |
| Best Optimizer       | 85.20%      | 2.66% | 88.20%        | 637.86 s      |
| Best Hyperparameters | 85.90%      | 1.62% | 89.00%        | 670.86 s      |
| Fine-Tuned Model     | 80.60%      | 2.06% | 89.60%        | 1769.49 s     |

**Discussion of Overall Results:** Results show it doesn't always mean single accuracy is the only criteria for selection of a model. It depends on various other factors as well. C1 gave us the most effective balance between cross validation acc and low variability it had low training cost also. Hence that's the best over here.

## 14. Conclusion

**Conclusion:** The good model always doesn't only select a strong architecture and training in once. The Mobilenetv2 (model) had a different behavior depending on the optimizer and various other factors. The regularization, the training strategy also contributed to the change of accuracy. C1 was the most balanced in our case which gave us accuracy of 89 percent total and F1 score of 88.8 percent which showed that the pretrained model generalized reasonably well to unseen images. Fine tuning required a much longer time than regular and didn't much difference in our case as the c1 was much better with this data.

This experiment had the value of comparisons of model based on different strategies and also seeing trade offs to choose between depending upon the computation size and our problem needs. A configuration with very high validation accuracy is not automatically the best it should behave consistently and remain practical.

## 15. Source code

[github.com/ssukesh/DeeplearningLab](https://github.com/ssukesh/DeeplearningLab)